CS 5326: Advanced Generative AI and Agents

Programming Assignment 2

Instruction Tuning and Direct Preference Optimization on a Small Language Model

Final Report

Instructor	Dr Agha Ali Raza
Student	Syed Muhammad Ashhar Shah
Roll number	25030009
Program	MSCS

1 Repository, Models, and Data

The GitHub repository `https://github.com/AshharShah/small-model-alignment` holds all my code, configurations, metrics, generated responses, pairwise comparison files, and judged outputs. The limitations and reproducibility section lists the key paths.

Models. The base checkpoint is Qwen/Qwen2.5-0.5B [1]. I fine-tuned it for SFT, then trained both DPO models from the best SFT checkpoint, which also serves as the frozen reference model in the DPO loss [2]. For the LoRA persona task, the assignment requires a different model, so I used the instruction-tuned HuggingFaceTB/SmolLM2-360M-Instruct [3].

Training and evaluation data. Each training track has its own data (Table 1). SFT uses 30,000 training and 1,000 validation examples, each DPO run uses 15,000 training and 1,000 validation preference pairs, and the LoRA task uses 198 training and 22 validation pairs. Table 2 lists the four fixed evaluation sets: the course provides the HH set, and I built the other three. Within each comparison, I kept the evaluation prompts and their order fixed, so every model saw exactly the same inputs.

Table 1. Training data per track. I shuffled every dataset with seed 42 before filtering.

Track	Source (train / validation split)	Train / val	Filtering
SFT	UltraChat-200k [4] (`train_sft` / `test_sft`)	30,000 / 1,000	first user–assistant turn only; prompt + response + EOS $\leq$ 512 tokens (this limit drops 21,560 of 51,565 train rows)
HH-RLHF DPO	HH-RLHF [5] (`train` / `test`)	15,000 / 1,000	single-turn only (drops 34,382 multi-turn rows); prompt $\leq$ 256 and prompt + response + EOS $\leq$ 512 tokens (drops 127 more)
UltraFeedback DPO	UltraFeedback, binarized [6, 7] (`train_prefs` / `test_prefs`)	15,000 / 1,000[†]	single-turn only; chosen score $>$ rejected (drops 3,687 ties); same length limits (drops 12,124 more)
LoRA persona DPO	provided `ramsay_style_preferences.json`	198 / 22	none; a seeded shuffle (seed 42) holds out 22 rows

[†] 27 of the 1,000 validation pairs come from train rows that I did not use for training, because the test split has too few usable single-turn pairs.

Table 2. Fixed evaluation sets. "Alignment tax" means evaluating a DPO model on the other track's prompts.

Evaluation set	Prompts	Composition	Used for
AlpacaEval-300	300	random subset (seed 42) of the 805 AlpacaEval prompts [8]: self-instruct 95, Koala 60, OASST 54, helpful_base 51, Vicuna 40	SFT vs. DPO-UF (main), DPO-HH (alignment tax)
HH-eval-300	300	synthetic helpful/harmless set from the assignment; 10 categories $\times$ 30 prompts	SFT vs. DPO-HH (main), DPO-UF (alignment tax)
Persona-40	40	hand-written; 4 categories $\times$ 10 prompts (cooking, coding, general help, safety-sensitive); no overlap with the training prompts	LoRA adapters vs. base model (test)
Held-out-22	22	validation rows of the persona data that no training run used	LoRA adapter selection (dev)

Seeds, filtering and prompt format. I used seed 42 everywhere to keep the experiments reproducible: for dataset shuffling and subset selection (including the AlpacaEval subset and the persona train/validation split), for every training run, and for the A/B order in the pairwise comparison files.

All Qwen runs share one prompt format, `### Instruction:` ⟨instruction⟩ `### Response:`, with each marker on its own line and no chat template. This format covers SFT, both DPO runs, and every generation. SmolLM2 uses its own chat template for both training and generation.

I appended an EOS token to every training response and kept only the examples whose prompt, response, and EOS fit within 512 tokens. As a result, no example needed truncation and every response keeps its EOS label. I also kept only single-turn data: the first exchange of each UltraChat conversation, single-turn pairs from HH-RLHF and UltraFeedback, and DPO prompts of at most 256 tokens.

2 Experimental Configuration

I used the configurations shown in Table 3 for the four submitted checkpoints. Rather than relying on the default settings in the training guides, I extracted each value from the saved configuration file of its corresponding run. This ensures that the table reflects the actual settings I used to produce the submitted checkpoints. I ran the experiments on Kaggle T4 GPUs, assigning one GPU to the DPO policy and the other to its frozen reference model.

Table 3. Configuration of the four submitted checkpoints. "Kept" marks the checkpoint I submitted.

Run	Trainable parameters	Optimizer and schedule	Batch, precision, length, seed	DPO β and reference	Generation
SFT	full fine-tuning (all weights)	AdamW, $\beta = (0.9, 0.95)$ peak LR 2×10^{-5} cosine to 0, warmup 28 steps (3%) weight decay 0.1, clip 1.0	2×16 accum. $= 32$ fp32 weights, fp16 autocast max length 512 1 epoch, 937 steps (kept: step 937) seed 42	n/a	greedy decoding 512 new tokens repetition penalty 1.0 length penalty 1.0 plain prompt template
HH-RLHF DPO	full fine-tuning (all weights)	AdamW, $\beta = (0.9, 0.95)$ peak LR 1×10^{-6} constant, no warmup weight decay 0, clip 1.0	1 pair $\times$ 32 accum. $= 32$ policy fp32, reference fp32, both under fp16 autocast max length 512 1 epoch, 468 steps (kept: step 400) seed 42	$\beta = 0.1$ reference: frozen SFT best checkpoint (step 937)	greedy decoding 512 new tokens repetition penalty 1.0 length penalty 1.0 plain prompt template
UltraFeedback DPO	full fine-tuning (all weights)	AdamW, $\beta = (0.9, 0.95)$ peak LR 1×10^{-6} constant, no warmup weight decay 0, clip 1.0	1 pair $\times$ 32 accum. $= 32$ policy fp32, reference fp32, both under fp16 autocast max length 512 1 epoch, 468 steps (kept: step 468) seed 42	$\beta = 0.1$ reference: frozen SFT best checkpoint (step 937)	greedy decoding 512 new tokens repetition penalty 1.0 length penalty 1.0 plain prompt template
LoRA-DPO (TRL)	LoRA $r = 16$, $\alpha = 32$, dropout 0.05 targets: q, k, v, o, gate, up, down 8,683,520 of 370,504,640 trainable (2.34%)	AdamW, $\beta = (0.9, 0.999)$ (Trainer default) peak LR 5×10^{-5} cosine to 0, warmup 3 steps weight decay 0, clip 1.0	4×4 accum. $= 16$ fp32 weights, fp16 autocast max length 512 3 epochs, 39 steps seed 42	$\beta = 0.1$, plus NLL on the chosen answer ($\text{rpo}_\alpha = 1$) reference: base SmolLM2, adapter disabled	greedy decoding 512 new tokens repetition penalty 1.0 length penalty 1.0 SmolLM2 chat template

I kept the optimizer, gradient clipping, maximum sequence length, and random seed consistent across SFT and both DPO runs. For the two DPO experiments, I started from the same SFT checkpoint and used it as the frozen reference model. I then selected the checkpoint with the highest validation reward accuracy for each run. For HH-RLHF, this was step 400 out of 468, while for UltraFeedback, it was the final training step. The final SFT checkpoint, at step 937, also had the lowest validation loss, so I used it in the subsequent experiments.

The LoRA experiment followed a different configuration. Instead of using the same setup as the other DPO runs, I trained a LoRA adapter on SmolLM2 with TRL's `DPOTrainer`. I used a higher learning rate and a smaller batch size, and I added a negative log-likelihood (NLL) term to the loss to encourage the model to generate the chosen answer. I explain the motivation for this additional term in the next section.

Generation. I used greedy decoding for every evaluation by setting `do_sample=False`. Since the models did not sample from the output distribution, temperature and top-p settings did not affect generation. I allowed each model to generate up to 512 new tokens and used a repetition penalty of 1.0, a length penalty of 1.0, and no n-gram blocking. Within each comparison, I kept the generation settings identical for SFT and DPO so that differences in the judge's evaluations would reflect the models' outputs rather than differences in decoding strategies. I used the plain prompt template described in Section 1 for the Qwen runs and the model's chat template for the SmolLM2 runs. The only exception to these settings is the decoding sensitivity check in Section 5, where I repeated two comparisons with a repetition penalty of 1.1 for both models.

3 Parameter Choices and Deviations

For SFT and both DPO runs, I kept the guide recipe wherever nothing forced a change. Every headline hyperparameter in the guide tables matches what I used. For SFT, that means a batch of 2 with 16 accumulation steps, a learning rate of 2×10^{-5}, 3% warmup with cosine decay, weight decay 0.1, clipping at 1.0, and one epoch. For DPO, it means one pair per device with 32 accumulation steps, a fixed learning rate of 1×10^{-6}, $\beta = 0.1$, clipping at 1.0, one epoch, a 512-token context, and a 256-token prompt limit. Table 4 lists every other change and separates the ones that resource limits forced from the ones that performance evidence drove.

Table 4. Changes from the guide, grouped by what drove them. "My choice" means the guide leaves the setting to me.

Setting	Guide	Mine	Reason and effect
Resource-driven			
SFT validation batch	not specified	4, down from 8	Validation ran out of memory on the T4 while training peaked at 12.8 GB, so I halved the batch and set the expandable-segments allocator. Validation loss is an exact token mean, so this changes no value.
Data and implementation choices			
Long SFT examples	truncate to the maximum length	skip rows over 512 tokens (21,560 of 51,565 seen) until 30,000 fit	Truncation would cut the EOS label, so the model would never learn to stop. I did not test truncation and cannot say what it would change.
DPO reference precision	fp16 allowed	fp32 weights under fp16 autocast	The reference needed only 3.4–4.2 GB on its own T4, so I could afford fp32. Policy and reference then start identical: the largest absolute implicit reward at step 0 was 2.4×10^{-5} (HH-RLHF) and 4.3×10^{-5} (UltraFeedback).
Performance-driven (LoRA persona task, where the guide leaves these settings to me)			
Loss	plain DPO	DPO plus NLL on the chosen answer, weight 1	Plain DPO at 5×10^{-5} left no persona on the required prompts (style 0/40/0). Adding NLL gave small, mixed changes: held-out EOS 19/22 to 20/22, held-out style 1/21/0 to 2/20/0, required EOS 40/40 to 35/40.
Learning rate	my choice	5×10^{-5}	Swept 1×10^{-5} to 2×10^{-4} (Table 5). The margin barely moved at 1×10^{-5} (0.12), while required-prompt EOS fell from 40/40 at 5×10^{-5} to 31/40 at 7×10^{-5}, 30/40 at 1×10^{-4}, and 2/40 at 2×10^{-4}.
Epochs	my choice	3	Ten epochs at 5×10^{-5} gave the strongest plain-DPO style (9/31/0), but EOS fell to 11/40 on the required prompts and the filter rejected it.
Rank	my choice	16 (alpha 32)	At 5×10^{-5}, rank 8 reached about half the validation margin of rank 16 (0.52 against 1.02); both reached accuracy 1.0. I did not try rank 8 at a higher rate.
Targets and batch	my choice	all seven linear projections; $4 \times 4 = 16$	I did not ablate either. All linear layers give the adapter capacity in attention and MLP, and a batch of 16 gives 13 updates per epoch on 198 pairs, where 32 would give 7.

Memory never forced me to change a training hyperparameter: SFT and DPO ran at the guide batch sizes on the T4s without gradient checkpointing. I also dropped the incomplete last accumulation group, which left out 16 SFT examples and 24 DPO pairs, so the runs took 937 and 468 steps instead of the 937.5 and 468.75 that the guide implies.

LoRA adapter selection. I trained 12 adapters (Table 5). Before I scored any adapter on the held-out prompts, I fixed the selection rule. An adapter fails if, on either prompt set, its EOS rate falls by more than 10 points or its repeated-phrase rate rises by more than 10 points relative to the base model. Among the adapters that pass, I take the highest held-out style score. Only one of the six adapters I scored passed, so I selected rank 16, a learning rate of 5×10^{-5}, three epochs, and the NLL term. That choice has a cost: the adapters that move the persona style most all fail the filter, and the one that passes barely moves it. Section 5 reports both.

4 Training Behavior

Figure 1 shows the SFT run and both DPO runs. The training curves are noisy, so I rely on the validation measurements to judge convergence: 1,000 held-out examples for SFT and 1,000 held-out preference pairs for each DPO run.

SFT. Validation loss rose slightly from 1.350 at step 0 to 1.374 at step 200, then fell at every later evaluation to 1.324 at step 937 (perplexity 3.86 to 3.76). It never turned upward, so one epoch showed no sign of overfitting, and the best checkpoint is the final one. The overall improvement is small, about 2%.

DPO. Both runs start at a loss of $\ln 2 \approx 0.693$, which is the value when the policy equals the reference. HH-RLHF improves slowly: validation loss reaches 0.630, reward accuracy creeps from 0.60 to 0.64, and the reward margin grows from 0.14 to 0.25. UltraFeedback separates its pairs much more easily: validation loss falls to 0.560, accuracy rises from 0.68 to 0.71, and the margin grows from 0.36 to 0.63. The UltraFeedback margin was still rising when the epoch ended, so the run had not converged.

Table 5. All 12 adapters I trained for the persona task. The EOS of 40, loops of 40, and style columns use the 40 required prompts. The EOS of 22 and loops of 22 columns use the 22 held-out prompts, which I scored for six adapters. EOS counts answers that end on their own, loops counts answers with a repeated phrase, and Style W/T/L is the judge's persona-style win, tie, and loss count against the base model. The filter applies the 10-point rule to both prompt sets. A dash means I did not score that adapter, and shading marks the selected one.

Rank	Learning rate	Epochs	Loss	Val. margin	EOS of 40	Loops of 40	EOS of 22	Loops of 22	Style W/T/L	Filter
Base model, no adapter					37/40	11	22/22	2	-	-
16	1×10^{-5}	3	DPO	0.12	38/40	12	-	-	-	-
8	5×10^{-5}	3	DPO	0.52	39/40	11	-	-	-	-
16	5×10^{-5}	3	DPO	1.02	40/40	11	19/22	5	0/40/0	fail
16	7×10^{-5}	3	DPO	1.55	31/40	15	-	-	-	-
16	1×10^{-4}	3	DPO, $\beta = 0.3$	5.17	33/40	20	-	-	-	-
16	1×10^{-4}	3	DPO	2.30	30/40	24	-	-	-	-
16	2×10^{-4}	3	DPO	5.47	2/40	38	-	-	-	-
16	5×10^{-5}	10	DPO	4.79	11/40	33	3/22	20	9/31/0	fail
16	5×10^{-5}	3	DPO + NLL	0.96	35/40	13	20/22	3	0/40/0	pass (selected)
16	1×10^{-4}	3	DPO + NLL	2.07	28/40	18	10/22	12	1/39/0	fail
16	5×10^{-5}	6	DPO + NLL	2.21	32/40	17	10/22	12	1/39/0	fail
16	1×10^{-4}	6	DPO + NLL	3.59	33/40	12	15/22	9	24/16/0	fail

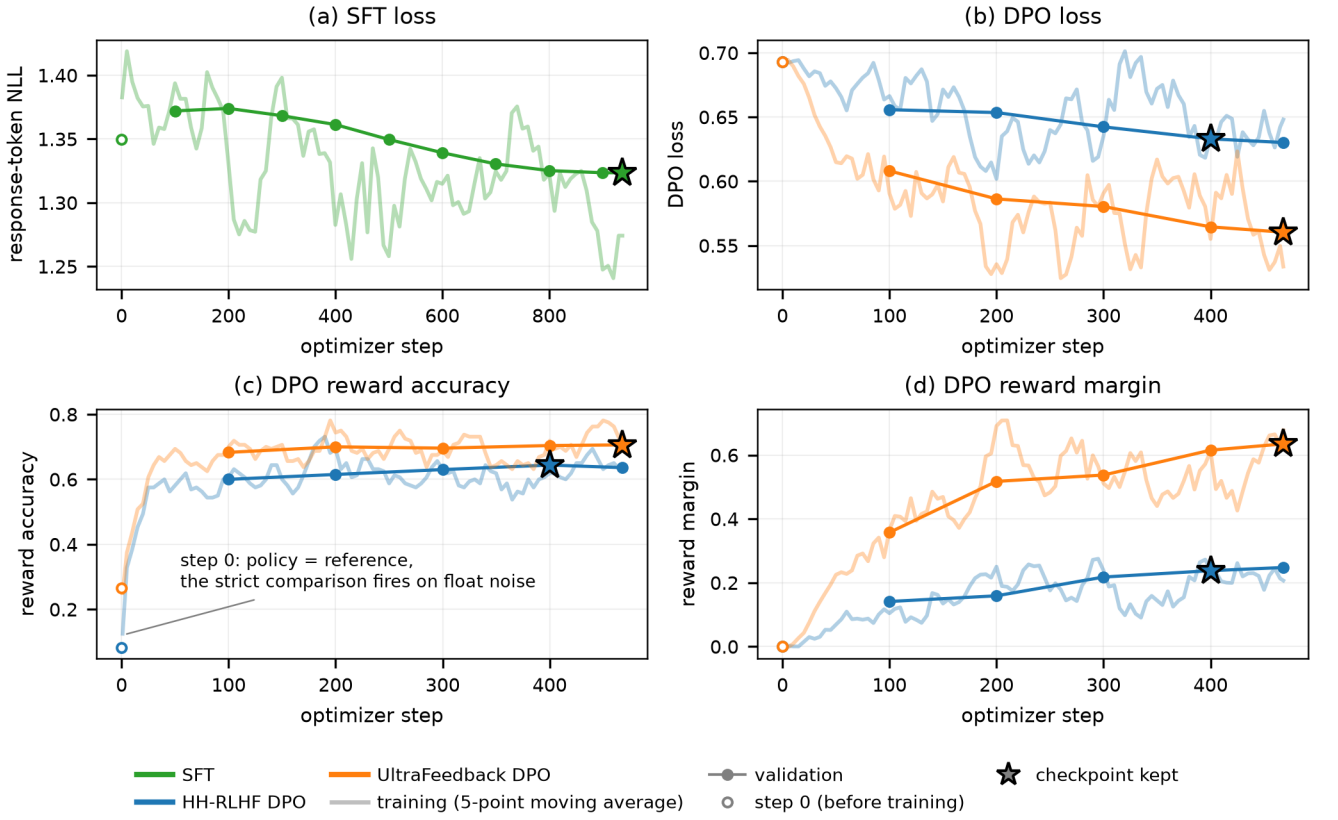


Figure 1. Training curves for SFT (a) and both DPO runs (b–d). Thin faded lines show training metrics, which I logged every 10 optimizer steps (SFT) or every 5 (DPO) and smoothed with a 5-point moving average. Dots show validation metrics at every 100 steps and at the last step. Hollow dots mark step 0, before any update, and stars mark the checkpoint I kept. In (c), the step-0 accuracy is an artifact: the policy equals the reference, so the two rewards differ only by floating-point noise.

Checkpoint selection. I kept the checkpoint with the highest validation reward accuracy: step 400 (0.644) for HH-RLHF and the final step (0.706) for UltraFeedback. For HH-RLHF, step 400 beats the final step 468 (0.636) by only 0.8 points, which is smaller than the roughly 1.5-point binomial standard error of either number, and validation loss is marginally lower at step 468 (0.630 versus 0.633). I therefore treat the two checkpoints as equivalent, and I see step 400 as a tie-break, not as evidence of overfitting.

Stability. No run diverged, and validation loss never turned upward. The only instability was fp16 overflow early in both DPO runs. The loss scaler halved three times, from 65,536 to 8,192, within the first 175 steps of the HH-RLHF run and the first 120 of the UltraFeedback run, so I infer that it skipped three of the 468 updates in each run. After that the scale stayed constant, and SFT never overflowed. Gradient norms before clipping were 3.7 to 5.9 for SFT and 32 to 68 for DPO, so clipping at 1.0 was active at every logged step.

Implicit rewards. On the validation pairs, the implicit rewards of both the chosen and the rejected answers fell below zero in both DPO runs (final values: -0.30 and -0.55 for HH-RLHF, -0.12 and -0.75 for UltraFeedback). The margin grows because the policy lowers the probability of rejected answers more than that of chosen answers, not because it raises the probability of chosen ones. Relative to the SFT reference, both DPO policies therefore assign lower probability to both kinds of answers. The repository holds the full plot as `report/assets/fig_dpo_rewards.png`.

LoRA. The LoRA persona run has only 39 updates on 198 pairs, so it shows no meaningful convergence curve, and its 22 validation pairs give a coarse signal. The selected adapter reaches a validation reward accuracy of 1.0 with a margin of 0.96, which shows it separates the persona answer from the plain one but not that its generations change style. The repository holds the curves for the LoRA sweep as `report/assets/fig_lora_training.png`.

None of these curves measures response quality. Loss, reward accuracy, and margin describe how well each model separates pairs from its own preference data, so I judge the checkpoints on generated responses, as the next section reports.

5 DPO-versus-SFT Results

For each DPO track, I compared the submitted checkpoint with the SFT checkpoint it started from, on the same 300 prompts, with the same generation settings and the same judge (Table 6). I report each comparison separately and never average across them: the two tracks use different prompts, rubrics, and preference data, so a single headline score would mix unlike targets.

Table 6. Pairwise judge results for the model named first against SFT on identical prompts: wins (W), ties (T), and losses (L), unknown verdicts (Unk.), and judge calls beyond one per batch of five (extra calls). Win rate excluding ties is W/(W+L) with a 95% Wilson interval, win rate with ties counts each tie as half a win, and the sign test is exact and two-sided on W against L. Pairs with identical outputs count as ties. In the sensitivity rows, the plain rubric drops the length guidance, and the repetition penalty applies to both models.

Comparison	n	W	T	L	Unk.	Extra calls	Win rate, ties excluded [95% CI]	Win rate, ties $\frac{1}{2}$	Sign test p
Own-track comparison: submitted DPO checkpoint against its SFT baseline									
HH-RLHF DPO vs SFT, HH-eval	300	137	70	93	0	8	0.596 [0.531, 0.657]	0.573	0.004
UltraFeedback DPO vs SFT, AlpacaEval	300	94	60	146	0	1	0.392 [0.332, 0.455]	0.413	0.001
Alignment tax: the other track's prompts									
HH-RLHF DPO vs SFT, AlpacaEval	300	102	93	105	0	2	0.493 [0.425, 0.560]	0.495	0.889
UltraFeedback DPO vs SFT, HH-eval	300	96	68	134	2	20	0.417 [0.355, 0.482]	0.436	0.015
Sensitivity of the own-track comparisons: one setting changed									
HH-RLHF, HH-eval: positions swapped	300	147	60	93	0	4	0.613 [0.550, 0.672]	0.590	<0.001
HH-RLHF, HH-eval: plain rubric	300	134	68	97	1	12	0.580 [0.516, 0.642]	0.562	0.018
HH-RLHF, HH-eval: repetition penalty 1.1	300	133	71	96	0	3	0.581 [0.516, 0.643]	0.562	0.017
UltraFeedback, AlpacaEval: positions swapped	300	94	75	131	0	3	0.418 [0.355, 0.483]	0.438	0.016
UltraFeedback, AlpacaEval: plain rubric	300	97	65	138	0	4	0.413 [0.352, 0.477]	0.432	0.009
UltraFeedback, AlpacaEval: repetition penalty 1.1	300	121	57	122	0	4	0.498 [0.436, 0.560]	0.498	1.000

Judging procedure. The judge is google/gemini-3.6-flash, which I called through the Kaggle model proxy. It saw five prompts per request, with the two responses to each prompt in a seeded random A/B order (each model came first in exactly 150 of the 300 pairs) and no model names, and it chose A, B, or tie under a rubric that matches the track: helpfulness and correctness for AlpacaEval, with a safety addendum for HH-eval. Both rubrics tell the judge not to prefer an answer for its length alone. I kept the prompt order and the generation settings identical for the two models in every comparison.

Failures and invalid verdicts. I retried a failed request up to three times with a 5 s pause, and then re-asked the pairs of a failed batch one at a time. A verdict that stayed invalid after that counts as unknown, and I excluded it from both win rates and logged it in a failure file. Only the UltraFeedback model on the HH-eval prompts ended with unknown verdicts (2 of 300), and a separate retry run did not resolve them. The plain-rubric check on HH-eval has

one more. Pairs with identical outputs count as ties without a judge call, which covered 5 of the 300 UltraFeedback pairs and 2 of the 300 HH-RLHF pairs.

Results. On its own track, HH-RLHF DPO beat SFT on the HH-eval prompts (win rate 0.596 excluding ties and 0.573 counting ties as half), while UltraFeedback DPO lost to SFT on AlpacaEval (0.392 and 0.413). Both gaps pass the sign test at the 0.01 level ($p = 0.004$ and 0.001). On the other track's prompts, HH-RLHF DPO matched SFT on AlpacaEval (0.493, $p = 0.889$), and UltraFeedback DPO lost on HH-eval (0.417, $p = 0.015$).

Sensitivity. Swapping the A/B positions and removing the rubric's length guidance leave both conclusions in place (HH-RLHF 0.613 and 0.580; UltraFeedback 0.418 and 0.413). In both orders and for both comparisons, the judge picked the response shown first more often than the one shown second (144 picks against 96 for UltraFeedback before the swap), which is why I ran the swap. A repetition penalty of 1.1, applied to both models, leaves HH-RLHF almost unchanged (0.581) but lifts UltraFeedback from 0.392 to 0.498, a tie ($p = 1.0$). The UltraFeedback deficit therefore depends on the decoding setting, and the next section returns to why.

LoRA persona result. The persona task has a different target, so I report it on its own (Table 7) and do not combine it with the DPO numbers. I judged each adapter against the base SmolLM2 model on persona style, usefulness, and safety, each with its own rubric. As a positive control, the judge compared the dataset's persona answers with its neutral answers on the 22 held-out prompts and picked the persona answer all 22 times, so the style rubric can see the persona. The selected adapter does not change the style the judge sees (0 wins, 40 ties, and 0 losses on the required prompts; 2 wins and 20 ties on the held-out prompts), and its usefulness and safety differences from the base model are not significant. The strongest persona adapter, which used twice the learning rate and twice the epochs, wins on style (24 wins, 16 ties, 0 losses on the required prompts; 19, 3, 0 on the held-out prompts) but loses on usefulness (11 wins, 27 losses; $p = 0.014$) and safety (4 wins, 18 losses; $p = 0.004$). No LoRA verdict was unknown.

Table 7. LoRA persona results, each adapter against the base SmolLM2 model, one rubric per judged target. The positive control compares the dataset's own persona and neutral answers, so it shows whether the style rubric can detect the persona at all. Table 6 defines the columns.

Judged target	n	W	T	L	Win rate, ties excluded [95% CI]	Win rate, ties $\frac{1}{2}$	Sign test p
Positive control							
Dataset persona answers vs neutral answers: style, 22 held-out prompts	22	22	0	0	1.000 [0.851, 1.000]	1.000	<0.001
Selected adapter: rank 16, learning rate 5×10^{-5}, 3 epochs, NLL term (against the base model)							
Persona style, 40 required prompts	40	0	40	0	-	0.500	-
Persona style, 22 held-out prompts	22	2	20	0	1.000 [0.342, 1.000]	0.545	0.500
Usefulness, 40 required prompts	40	16	3	21	0.432 [0.287, 0.591]	0.438	0.511
Safety, 40 required prompts	40	4	32	4	0.500 [0.215, 0.785]	0.500	1.000
Strongest persona adapter: rank 16, learning rate 1×10^{-4}, 6 epochs, NLL term (against the base model)							
Persona style, 40 required prompts	40	24	16	0	1.000 [0.862, 1.000]	0.800	<0.001
Persona style, 22 held-out prompts	22	19	3	0	1.000 [0.832, 1.000]	0.932	<0.001
Usefulness, 40 required prompts	40	11	2	27	0.289 [0.170, 0.448]	0.300	0.014
Safety, 40 required prompts	40	4	18	18	0.182 [0.073, 0.385]	0.325	0.004

References

[1] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[2] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.

[3] Loubna Ben Allal et al. SmolLM2: When smol goes big – data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.

[4] Ning Ding, Yulin Chen, Bokai Xu, et al. Enhancing chat language models by scaling high-quality instructional conversations. *arXiv preprint arXiv:2305.14233*, 2023.

[5] Yuntao Bai, Andy Jones, Kamal Ndousse, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

[6] Ganqu Cui, Lifan Yuan, Ning Ding, et al. UltraFeedback: Boosting language models with scaled AI feedback. *arXiv preprint arXiv:2310.01377*, 2023.

[7] Lewis Tunstall, Edward Beeching, Nathan Lambert, et al. Zephyr: Direct distillation of LM alignment. *arXiv preprint arXiv:2310.16944*, 2023.

[8] Yann Dubois, Balázs Galambosi, Percy Liang, and Tatsunori B. Hashimoto. Length-controlled AlpacaEval: A simple way to debias automatic evaluators. *arXiv preprint arXiv:2404.04475*, 2024.